

40 Bài tập Chuỗi ký tự (Cơ bản - Trung bình)

Nhóm 1: Nhập, Xuất và Thuộc tính cơ bản

Yêu cầu sinh viên viết hàm cho mỗi bài, ví dụ: void nhapChuoi(char s[]), int tinhChieuDai(char s[]).

1. **Nhập chuỗi:** Viết hàm nhập một chuỗi, cho phép nhập cả khoảng trắng (sử dụng fgets thay vì scanf("%s")).
2. **Xuất chuỗi:** Viết hàm xuất chuỗi đã nhập.
3. **Tính chiều dài:** Viết hàm int my_strlen(char s[]) tính và trả về độ dài chuỗi (không dùng strlen).
4. **Đếm nguyên âm:** Viết hàm đếm số lượng nguyên âm (a, e, i, o, u, cả hoa và thường) trong chuỗi.
5. **Đếm phụ âm:** Viết hàm đếm số lượng phụ âm trong chuỗi.
6. **Đếm khoảng trắng:** Viết hàm đếm số lượng ký tự khoảng trắng trong chuỗi.
7. **Đếm chữ số:** Viết hàm đếm số lượng chữ số (0-9) trong chuỗi (dùng isdigit).
8. **Đếm ký tự đặc biệt:** Viết hàm đếm số lượng ký tự không phải là chữ cái, chữ số, hoặc khoảng trắng.

Nhóm 2: Thao tác với ctype.h

9. **Chuyển sang in hoa:** Viết hàm void chuyenInHoa(char s[]) chuyển tất cả ký tự thường thành hoa (dùng toupper).
10. **Chuyển sang in thường:** Viết hàm void chuyenInThuong(char s[]) chuyển tất cả ký tự hoa thành thường (dùng tolower).
11. **Nghịch đảo case:** Viết hàm void nghichDaoCase(char s[]) chuyển ký tự hoa thành thường và ngược lại.
12. **Viết hoa ký tự đầu:** Viết hàm void hoaKyTuDau(char s[]) chỉ chuyển ký tự đầu tiên của chuỗi sang in hoa (nếu nó là chữ cái).
13. **Viết hoa chữ cái đầu mỗi từ:** Viết hàm void hoaDauTu(char s[]) chuyển ký tự đầu tiên của mỗi từ sang in hoa (ví dụ: "nguyen van a" -> "Nguyen Van A").

Nhóm 3: Tự triển khai thư viện string.h

Các bài tập này giúp sinh viên hiểu rõ bản chất của chuỗi. Yêu cầu không dùng các hàm string.h tương ứng.

14. **my_strcpy(char dest[], char src[]):** Tự viết hàm sao chép chuỗi src sang dest.
15. **my_strcat(char dest[], char src[]):** Tự viết hàm nối chuỗi src vào cuối chuỗi dest.
16. **my_strcmp(char s1[], char s2[]):** Tự viết hàm so sánh hai chuỗi s1 và s2. Trả về 0 (bằng nhau), < 0 (s1 < s2), > 0 (s1 > s2).
17. **my_strchr(char s[], char c):** Tự viết hàm tìm và trả về con trỏ đến vị trí xuất hiện đầu tiên của ký tự c trong s (hoặc trả về NULL nếu không tìm thấy).
18. **my_strstr(char s[], char sub[]):** Tự viết hàm tìm và trả về con trỏ đến vị trí xuất hiện đầu tiên của chuỗi con sub trong s (hoặc NULL).

19. **my_strcmp_case(char s1[], char s2[]):** Tự viết hàm so sánh 2 chuỗi không phân biệt hoa thường.

Nhóm 4: Bài toán Đếm, Tìm kiếm, Kiểm tra (Sử dụng string.h)

20. **Kiểm tra đối xứng (Palindrome):** Viết hàm int laChuoiDoiXung(char s[]) kiểm tra chuỗi có đối xứng hay không (ví dụ: "madam").
21. **Đếm số từ:** Viết hàm int demSoTu(char s[]) đếm số lượng từ trong chuỗi (các từ cách nhau bằng 1 hoặc nhiều khoảng trắng).
22. **Tìm từ đầu tiên:** Viết hàm void timTuDauTien(char s[], char word[]) trả về từ đầu tiên trong chuỗi.
23. **Tìm từ cuối cùng:** Viết hàm void timTuCuoiCung(char s[], char word[]) trả về từ cuối cùng trong chuỗi.
24. **Đảo ngược chuỗi (In-place):** Viết hàm void daoNguocChuoi(char s[]) đảo ngược chuỗi ngay trên chuỗi gốc (ví dụ: "abc" -> "cba").
25. **Tìm ký tự xuất hiện nhiều nhất:** Viết hàm char timKyTuMax(char s[]) trả về ký tự xuất hiện nhiều lần nhất trong chuỗi.
26. **Kiểm tra chuỗi con:** Viết hàm int coChuaChuoiCon(char s[], char sub[]) kiểm tra xem s có chứa sub hay không (dùng strstr).
27. **Kiểm tra số nguyên:** Viết hàm int laSoNguyen(char s[]) kiểm tra chuỗi có phải là một số nguyên hợp lệ hay không.
28. **Kiểm tra số thực:** Viết hàm int laSoThuc(char s[]) kiểm tra chuỗi có phải là một số thực hợp lệ hay không (chứa 1 dấu . và các chữ số).
29. **Nối 2 chuỗi an toàn:** Viết hàm void noiChuoiAnToan(char s1[], char s2[], int maxSize) nối s2 vào s1 nhưng đảm bảo s1 không vượt quá maxSize.

Nhóm 5: Thao tác Biến đổi chuỗi (In-place)

Đây là nhóm bài tập quan trọng, yêu cầu sinh viên thay đổi trực tiếp chuỗi đầu vào.

30. **Xóa ký tự c:** Viết hàm void xoaKyTu(char s[], char c) xóa tất cả các lần xuất hiện của ký tự c trong chuỗi.
31. **Xóa khoảng trắng thừa:** Viết hàm void xoaKhoangTrangThua(char s[]) xóa các khoảng trắng ở đầu, cuối chuỗi và các khoảng trắng lặp lại ở giữa.
32. **Chuẩn hóa tên riêng:** Viết hàm void chuanHoaTen(char s[]) kết hợp **Bài 31** và **Bài 13** (xóa khoảng trắng thừa và viết hoa chữ cái đầu mỗi từ).
33. **Thay thế ký tự:** Viết hàm void thayTheKyTu(char s[], char c1, char c2) thay thế tất cả ký tự c1 bằng c2.
34. **Xóa một chuỗi con:** Viết hàm void xoaChuoiCon(char s[], int viTri, int doDai) xóa một đoạn chuỗi con bắt đầu từ viTri và có doDai xác định.
35. **Chèn chuỗi:** Viết hàm void chenChuoi(char s1[], char s2[], int viTri) chèn chuỗi s2 vào s1 tại viTri.
36. **Sắp xếp ký tự:** Viết hàm void sapXepKyTu(char s[]) sắp xếp các ký tự trong chuỗi theo thứ tự alphabet (ví dụ: "cbade" -> "abcde").

37. **Tách họ:** Viết hàm void tachHo(char hoTen[], char ho[]) tách phần họ (từ đầu tiên) của chuỗi hoTen ra.
38. **Tách tên:** Viết hàm void tachTen(char hoTen[], char ten[]) tách phần tên (từ cuối cùng) của chuỗi hoTen ra.
39. **In các từ theo thứ tự ngược:** Viết hàm void inTuNguoc(char s[]) in các từ trong chuỗi theo thứ tự ngược lại (ví dụ: "Lap trinh C" -> "C trinh Lap").
40. **Xóa tất cả chữ số:** Viết hàm void xoaChuSo(char s[]) xóa tất cả các chữ số khỏi chuỗi.

10 Bài tập Chuỗi ký tự (Nâng cao)

Những bài tập này đòi hỏi logic phức tạp hơn, thường phải dùng strtok hoặc quản lý nhiều chuỗi/con trỏ.

41. **Tìm và thay thế chuỗi con (lần đầu):** Viết hàm void timVaThayThe(char s[], char sub_find[], char sub_replace[]) tìm và thay thế lần xuất hiện đầu tiên của sub_find bằng sub_replace.
42. **Tìm và thay thế tất cả:** Viết hàm void timVaThayTheTatCa(char s[], char sub_find[], char sub_replace[]) tìm và thay thế TẤT CẢ các lần xuất hiện của sub_find bằng sub_replace.
43. **Tìm từ dài nhất:** Viết hàm void timTuDaiNhat(char s[], char longestWord[]) tìm và trả về từ dài nhất trong chuỗi (sử dụng strtok để tách từ).
44. **Đảo ngược thứ tự các từ (In-place):** Viết hàm void daoNguocTu(char s[]) đảo ngược thứ tự các từ (ví dụ: "Lap trinh C" -> "C trinh Lap") nhưng thay đổi ngay trên chuỗi gốc.

Gợi ý: Đảo ngược toàn bộ chuỗi ("C hnirt paL"), sau đó đảo ngược lại từng từ ("C", "hnirt", "paL") -> "C trinh Lap".

45. **Tính tổng các số trong chuỗi:** Viết hàm int tinhTongCacSo(char s[]) tính tổng các số xuất hiện trong chuỗi.

Ví dụ: "abc 123 xyz 45 cde 10" -> \$123 + 45 + 10 = 178\$.

46. **Kiểm tra Email hợp lệ:** Viết hàm int laEmailHopLe(char s[]) kiểm tra chuỗi có phải là email hợp lệ hay không (theo quy tắc đơn giản: chỉ có một ký tự @, không có khoảng trắng, có ít nhất một dấu . sau @).
47. **Trích xuất Họ, Tên lót, Tên:** Viết hàm void tachHoTen(char s[], char ho[], char tenLot[], char ten[]) tách chuỗi "Nguyen Van An" thành 3 phần.
48. **Tìm chuỗi con đối xứng dài nhất:** Viết hàm void timChuoiDoiXungDaiNhat(char s[], char result[]) tìm và trả về chuỗi con (không nhất thiết là từ) dài nhất trong s mà đối xứng.

Ví dụ: "abracadabra" -> "aca". "banana" -> "anana".

49. Nén chuỗi đơn giản: Viết hàm void nenChuoi(char s[], char result[]) nén chuỗi theo số lần lặp.

Ví dụ: "AAABBBCCDAA" -> "A3B3C2D1A2".

50. Giải nén chuỗi đơn giản: Viết hàm void giaiNenChuoi(char s[], char result[]) giải nén chuỗi đã nén ở Bài 49.

Ví dụ: "A3B3C2D1A2" -> "AAABBBCCDAA".

Thư viện và Hàm hỗ trợ

Tất cả các bài giải dưới đây đều cần các thư viện sau:

C

```
#include <stdio.h>
#include <string.h> // Cho strlen, strstr, strcpy, memmove...
#include <ctype.h>  // Cho isdigit, isspace...
```

Lưu ý: Để chạy thử, bạn cần đặt các hàm này vào trong một tệp .c và tạo hàm main() để gọi chúng.

Lời giải 10 Bài tập Nâng cao

Bài 41: Tìm và thay thế chuỗi con (lần đầu)

Ý tưởng: Dùng strstr để tìm vị trí sub_find. Dùng memmove để dời phần chuỗi còn lại (cái đuôi) sang phải hoặc trái, tùy thuộc vào chênh lệch độ dài giữa sub_replace và sub_find. Cuối cùng, dùng memcpy để chép sub_replace vào.

```
void timVaThayThe(char s[], char sub_find[], char sub_replace[]) {
    char *pos = strstr(s, sub_find);

    // 1. Nếu không tìm thấy, thoát
    if (pos == NULL) {
        return;
    }

    int len_find = strlen(sub_find);
    int len_replace = strlen(sub_replace);

    // 2. Tính độ dài của phần đuôi (từ sau sub_find đến hết)
    // +1 để bao gồm cả ký tự null '\0'
    int len_tail = strlen(pos + len_find) + 1;
```

```

// 3. Dời phần đuôi
//     memmove an toàn cho các vùng nhớ chồng lấp
memmove(pos + len_replace, pos + len_find, len_tail);

// 4. Chép chuỗi thay thế vào vị trí
//     memcpy nhanh hơn strcpy vì ta đã biết rõ độ dài
memcpy(pos, sub_replace, len_replace);
}

```

Bài 42: Tìm và thay thế tất cả

Ý tưởng: Lặp lại **Bài 41**. Tuy nhiên, sau mỗi lần thay thế, con trỏ tìm kiếm phải được dời đến ngay sau chuỗi vừa thay thế (`pos + len_replace`) để tránh lặp vô tận (ví dụ: thay "a" bằng "ba").

```

void timVaThayTheTatCa(char s[], char sub_find[], char sub_replace[]) {
    // Nếu chuỗi tìm là rỗng, thoát để tránh lặp vô tận
    if (strlen(sub_find) == 0) {
        return;
    }

    char *pos = s; // Con trỏ bắt đầu tìm kiếm
    int len_find = strlen(sub_find);
    int len_replace = strlen(sub_replace);

    while ((pos = strstr(pos, sub_find)) != NULL) {
        // Tìm thấy tại vị trí 'pos'

        int len_tail = strlen(pos + len_find) + 1;

        // Dời phần đuôi
        memmove(pos + len_replace, pos + len_find, len_tail);

        // Chép chuỗi thay thế
        memcpy(pos, sub_replace, len_replace);

        // Dời con trỏ tìm kiếm đến ngay sau chuỗi vừa thay thế
        pos += len_replace;
    }
}

```

Bài 43: Tìm từ dài nhất

Ý tưởng: Dùng `strtok` để tách chuỗi thành các từ (token). Cần tạo một bản sao của chuỗi vì `strtok` sẽ làm thay đổi chuỗi gốc. Duyệt qua các từ và tìm từ dài nhất.

```

void timTuDaiNhat(char s[], char longestWord[]) {
    // Phải tạo bản sao vì strtok sẽ sửa chuỗi
    char temp_s[1000];
    strcpy(temp_s, s);

```

```

// Các ký tự phân cách từ
const char *delim = " \t\n\r,.;!()?";

char *token = strtok(temp_s, delim);

int maxLen = 0;
longestWord[0] = '\0'; // Khởi tạo chuỗi rỗng

while (token != NULL) {
    int currentLen = strlen(token);
    if (currentLen > maxLen) {
        maxLen = currentLen;
        strcpy(longestWord, token);
    }
    token = strtok(NULL, delim);
}
}

```

Bài 44: Đảo ngược thứ tự các từ (In-place)

Ý tưởng: Sử dụng thuật toán 2 bước:

1. Viết hàm `reverse(start, end)` để đảo ngược một phần chuỗi.
2. Đảo ngược *toàn bộ* chuỗi ("Lap trinh C" -> "C hnirt paL").
3. Duyệt chuỗi đã đảo, tìm từng từ ("C", "hnirt", "paL") và đảo ngược lại *chỉ từ đó* ("C", "trinh", "Lap").

```

// 1. Hàm hỗ trợ đảo một phần chuỗi (từ start đến end)
void reverse(char *start, char *end) {
    char temp;
    while (start < end) {
        temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

```

```

void daoNguocTu(char s[]) {
    int len = strlen(s);

    // 2. Đảo ngược toàn bộ chuỗi
    reverse(s, s + len - 1);

    // 3. Đảo ngược lại từng từ
    char *start = s; // Con trỏ đầu từ
    char *end = s;   // Con trỏ cuối từ

    while (*start) {
        // Bỏ qua khoảng trắng đầu
        while (*start && isspace(*start)) {
            start++;
        }
    }
}

```

```

    }
    if (*start == '\0') break; // Kết thúc chuỗi

    // end tìm cuối từ
    end = start;
    while (*end && !isspace(*end)) {
        end++;
    }

    // Đảo ngược từ hiện tại
    reverse(start, end - 1);

    // Di chuyển start đến vị trí của end
    start = end;
}

// Có thể cần chuẩn hóa lại khoảng trắng nếu muốn
// (Bài này chỉ yêu cầu đảo từ)
}

```

Bài 45: Tính tổng các số trong chuỗi

Ý tưởng: Duyệt chuỗi, dùng một biến cờ đangDocSo. Nếu gặp chữ số, bắt đầu cộng dồn vào currentNum. Nếu đang đọc số mà gặp ký tự không phải số (hoặc hết chuỗi), cộng currentNum vào totalSum và reset.

```

int tinhTongCacSo(char s[]) {
    int totalSum = 0;
    int currentNum = 0;
    int đangDocSo = 0; // Cờ hiệu
    int len = strlen(s);

    for (int i = 0; i <= len; i++) { // Duyệt đến cả ký tự null
        if (isdigit(s[i])) {
            // Nếu là số, cộng dồn
            currentNum = currentNum * 10 + (s[i] - '0');
            đangDocSo = 1;
        } else {
            // Nếu là ký tự khác (hoặc cuối chuỗi)
            if (đangDocSo) {
                // Kết thúc một số
                totalSum += currentNum;
                // Reset
                currentNum = 0;
                đangDocSo = 0;
            }
        }
    }
    return totalSum;
}

```

Bài 46: Kiểm tra Email hợp lệ (Đơn giản)

Ý tưởng: Dùng `strchr` để tìm `@` và `.` và `strrchr` để tìm `@` cuối cùng (nếu có nhiều).

```
int laEmailHopLe(char s[]) {
    char *at_pos = strchr(s, '@'); // Tìm '@' đầu tiên

    // 1. Không có '@'
    if (at_pos == NULL) return 0;

    // 2. '@' là ký tự đầu tiên
    if (at_pos == s) return 0;

    // 3. Có nhiều hơn 1 '@'
    if (strrchr(s, '@') != at_pos) return 0;

    // 4. Có khoảng trắng
    if (strchr(s, ' ') != NULL) return 0;

    // 5. Không có dấu '.' nào sau '@'
    char *dot_pos = strchr(at_pos + 1, '.');
    if (dot_pos == NULL) return 0;

    // 6. Dấu '.' ngay sau '@' (ví dụ: "a@.com")
    if (dot_pos == at_pos + 1) return 0;

    // 7. Dấu '.' là ký tự cuối cùng (ví dụ: "a@b.")
    if (*(dot_pos + 1) == '\0') return 0;

    return 1; // Đạt mọi điều kiện
}
```

Bài 47: Trích xuất Họ, Tên lót, Tên

Ý tưởng: Dùng `strchr` tìm khoảng trắng đầu tiên, `strrchr` tìm khoảng trắng cuối cùng.

- Họ: Từ đầu đến khoảng trắng đầu.
- Tên: Từ khoảng trắng cuối đến hết.
- Tên lót: Từ sau khoảng trắng đầu đến trước khoảng trắng cuối.

```
void tachHoTen(char s[], char ho[], char tenLot[], char ten[]) {
    // Khởi tạo rỗng
    ho[0] = '\0';
    tenLot[0] = '\0';
    ten[0] = '\0';

    char *first_space = strchr(s, ' ');
    char *last_space = strrchr(s, ' ');

    if (first_space == NULL) {
        // Không có khoảng trắng (ví dụ: "An")
        strcpy(ten, s);
    } else if (first_space == last_space) {
        // Chỉ có 1 khoảng trắng (ví dụ: "Van An")
        strncpy(ho, s, first_space - s);
```



```

        ho[first_space - s] = '\0';

        strcpy(ten, last_space + 1);
    } else {
        // Có nhiều hơn 1 khoảng trắng (ví dụ: "Nguyen Van An")
        // Tách họ
        strncpy(ho, s, first_space - s);
        ho[first_space - s] = '\0';

        // Tách tên
        strcpy(ten, last_space + 1);

        // Tách tên lót
        int len_lot = last_space - (first_space + 1);
        strncpy(tenLot, first_space + 1, len_lot);
        tenLot[len_lot] = '\0';
    }
}

```

Bài 48: Tìm chuỗi con đối xứng dài nhất

Ý tưởng: Sử dụng thuật toán "Mở rộng từ tâm". Với mỗi ký tự, ta coi nó là tâm của một chuỗi đối xứng:

1. Tâm là 1 ký tự (trường hợp độ dài lẻ, ví dụ: "ana").
2. Tâm là 2 ký tự (trường hợp độ dài chẵn, ví dụ: "anna"). Ta viết một hàm `expand` để mở rộng ra 2 bên và cập nhật độ dài lớn nhất.

```

// Hàm hỗ trợ mở rộng từ tâm
void expand(char s[], int left, int right, int *start, int *maxLen) {
    int len = strlen(s);
    while (left >= 0 && right < len && s[left] == s[right]) {
        int currentLen = right - left + 1;
        if (currentLen > *maxLen) {
            *maxLen = currentLen;
            *start = left;
        }
        left--;
        right++;
    }
}

void timChuoiDoiXungDaiNhat(char s[], char result[]) {
    int len = strlen(s);
    if (len <= 1) {
        strcpy(result, s);
        return;
    }

    int maxLen = 1;
    int start = 0;

    for (int i = 0; i < len; i++) {

```

```

        // 1. Tâm là 1 ký tự (trường hợp lẻ)
        expand(s, i, i, &start, &maxLen);

        // 2. Tâm là 2 ký tự (trường hợp chẵn)
        expand(s, i, i + 1, &start, &maxLen);
    }

    // Trích xuất kết quả
    strncpy(result, s + start, maxLen);
    result[maxLen] = '\0';
}

```

Bài 49: Nén chuỗi đơn giản (Run-Length Encoding)

Ý tưởng: Duyệt chuỗi, đếm các ký tự lặp lại. Dùng `sprintf` để chuyển số đếm (int) thành chuỗi và nối vào `result`.

```

void nenChuoi(char s[], char result[]) {
    int len = strlen(s);
    int i = 0; // Con trỏ duyệt s
    int j = 0; // Con trỏ ghi result

    while (i < len) {
        char currentChar = s[i];
        int count = 1;

        // Đếm số lần lặp
        while (i + 1 < len && s[i + 1] == currentChar) {
            count++;
            i++;
        }

        // Ghi ký tự vào result
        result[j++] = currentChar;

        // Ghi số đếm vào result (dùng sprintf)
        // sprintf trả về số ký tự đã ghi
        j += sprintf(result + j, "%d", count);

        i++; // Di chuyển đến ký tự tiếp theo
    }

    result[j] = '\0'; // Kết thúc chuỗi result
}

```

Bài 50: Giải nén chuỗi đơn giản

Ý tưởng: Duyệt chuỗi nén, khi gặp chữ cái thì lưu lại. Khi gặp chữ số, bắt đầu đọc số (có thể nhiều hơn 1 chữ số). Sau khi đọc xong số, lặp và ghi ký tự đã lưu vào `result`.

```

void giaiNenChuoi(char s[], char result[]) {

```

```
int len = strlen(s);
int i = 0; // Con trỏ duyệt s
int j = 0; // Con trỏ ghi result

while (i < len) {
    char charToRepeat = s[i];
    i++; // Di chuyển con trỏ đến phần số

    int count = 0;

    // Đọc số (vì có thể là 1, 10, 100...)
    while (i < len && isdigit(s[i])) {
        count = count * 10 + (s[i] - '0');
        i++;
    }

    // Lặp và ghi vào result
    for (int k = 0; k < count; k++) {
        result[j++] = charToRepeat;
    }
}

result[j] = '\\0'; // Kết thúc chuỗi result
}
```